AIオーケストレーションにおける性能維持と実行コスト削減のためのハーネス制御

性能向上に役立つ処理にも、費用がかかる

AIの能力と分担を活用して、複雑な仕事を任せたい。
取得・共有・実行・検証を増やす過程で、重複や不要な処理も生じ得る。

費用が積み上がる経路の例（研究する問題の模式図）

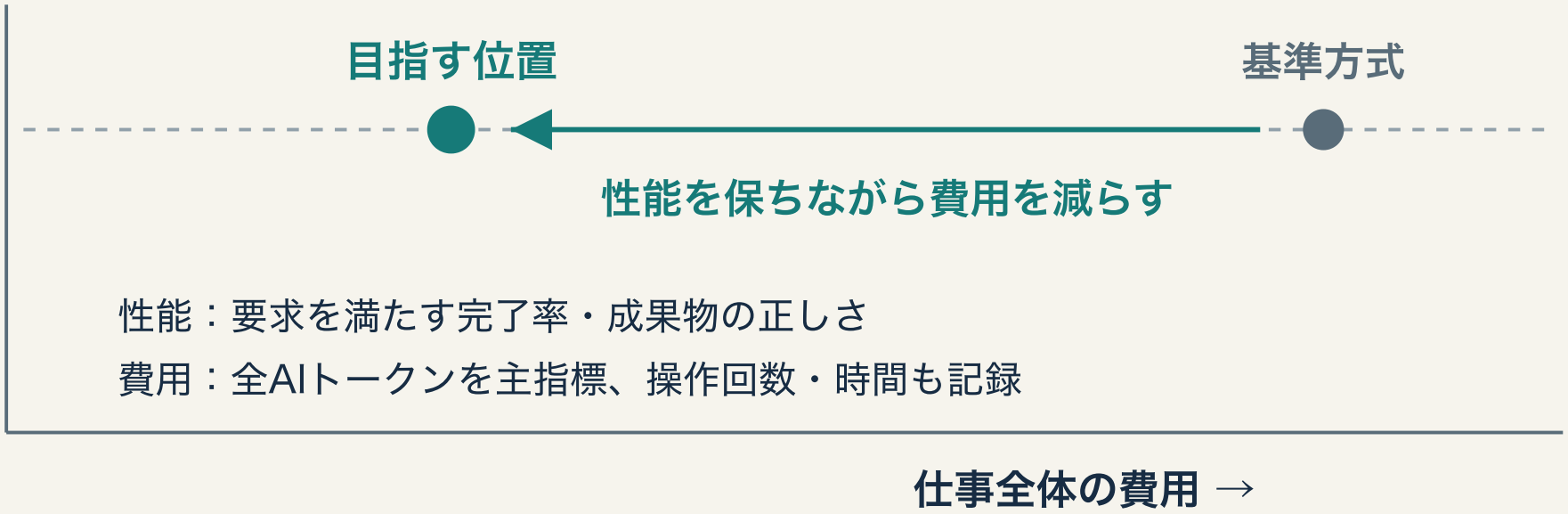


減らしたいのは、成果に必要な処理を除いた余分な費用。必要性の判定が研究課題。

性能を維持し、仕事全体の実行費用を減らす

ハーネスが必要な情報と処理を残し、不要な取得・共有・実行を削減する。

目標を表す概念図・実測ではない



性能を保てたかを確認してから、削減量を比べる

評価すること	確認する内容
タスクの性能	完了率・成果物の正しさを、同じ課題の強い基準と比較
総費用	取得・共有・制御・検証・失敗を含む、全入力＋全出力
削減の副作用	必要情報の欠落、再取得・再実行の増加、品質低下
維持と言える範囲	許容差と評価手順を事前に定め、ばらつきも確認

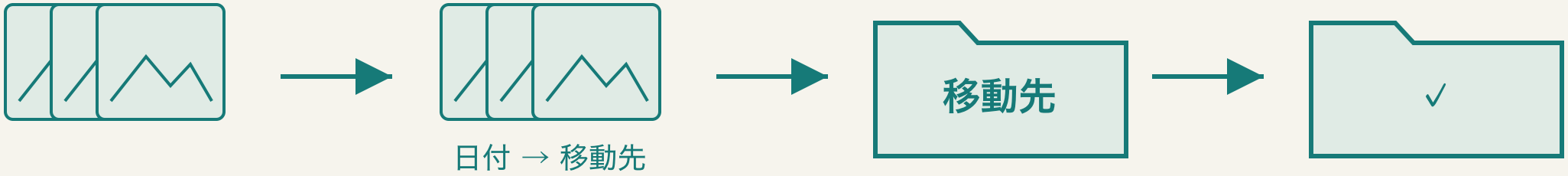
一回の入力や呼び出し回数が減っても、仕事全体が高くつけば削減ではない。

> 現在の実験は、費用を減らすつもりの操作が逆効果になる条件を調べる段階。

複数工程の仕事で、削減の影響を測る

たとえば写真整理では、方法を答えるだけでなく、
対象を調べて移動し、要求どおりに終えたか確かめる必要がある。

例：写真を撮影日ごとのフォルダーへ整理する



一覧を取得
何が対象か

撮影日を確認
どこへ移すか

写真を移動
実際に変更

完了を確認
要求を満たしたか

この間に、取得した値・判断・操作済みの内容をつなぐ必要がある。

作業が進むほど、引き継ぐ情報も増える

過去の応答や取得結果を毎回入力すると、入力が長くなる。
一部を省けば、作業の続きに必要な情報まで見えなくなる可能性がある。

説明用の場面：途中で会話履歴を短くして再開する

「一覧を取得した。撮影日で分類した。
次は、未処理の写真を移動する」



再開したAIに見えるのは直近の結果だけ
「残っている一覧は何に使うもの？」

値が残ること　　≠　　用途・作業の続きが伝わること

想定される支障：調べ直す／計算し直す／処理を抜かす／重複して処理する

AIの作業状態は、三つの場所に分かれる

「履歴が残る」「変数が残る」「操作の結果が残る」は、それぞれ違う。

① 会話履歴

AIの過去の応答

ツールが返した結果

次の入力としてAIに見せる

今回、全部／直近に変える

② Pythonの変数

取得した一覧や計算結果

実行環境にある値

コードで再利用できる

今回、保持／初期化に変える

③ アプリの状態

移動済みのファイル

送信済みの依頼や返金

すでに実行した操作の結果

全条件で同じ状態から開始

変数を消しても、すでに移動した写真は元に戻さない。

変数を残せば、会話履歴は減らせるか

期待する利点

取得・計算した値を再利用すれば、長い過去の会話を毎回渡す費用を減らせるかもしれない。

確かめる必要がある点

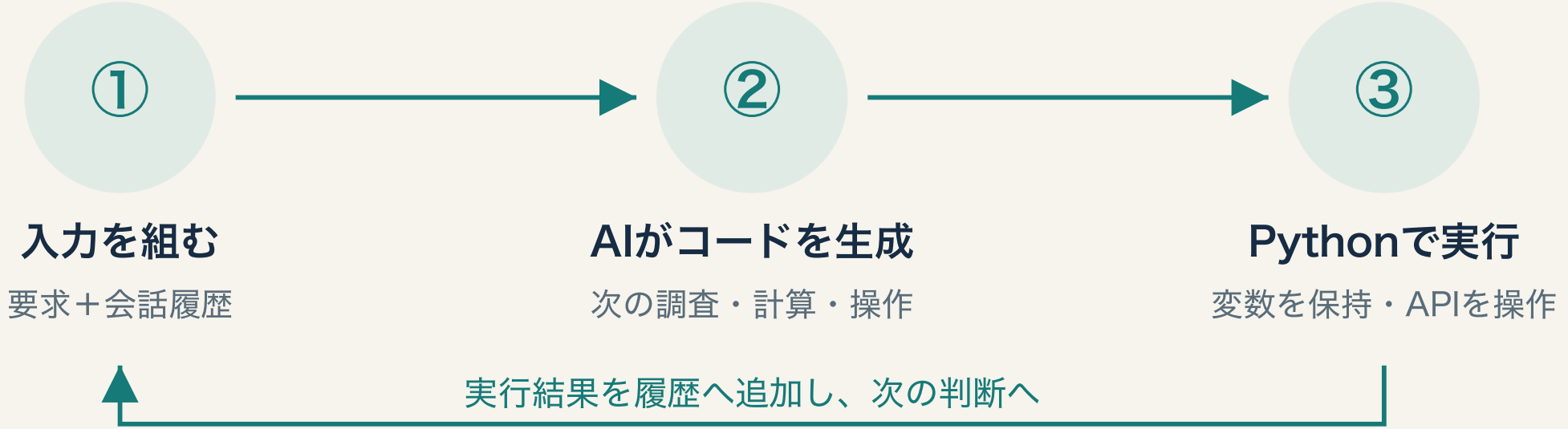
値だけでは、その用途、未完了の作業、使うAPIが伝わらず、調べ直しが増えるかもしれない。

> 全体目的との関係：入力を減らしても、性能低下や再取得で高くつかないかを測る。

ハーネスが、入力と作業状態を管理する

同じAIにPythonコードを生成させ、模擬アプリを操作する。
今回変更するのは、再開時に渡す会話と、Python状態の保持方法。

利用者の要求 → **ハーネス：次の入力と実行手順を管理するプログラム**



入力削減が、別の費用を増やしていないか調べる

全体の研究対象

情報取得・共有・呼び出し・実行・検証の必要性を制御

担当再編や結果再利用も、性能を保って費用を減らすための手段。

今、測っている一部分

担当を固定し、残す会話と変数を変える

入力を減らした結果、必要情報の再取得や失敗が増えないかを調べる。

> 未測定：複数担当間の重複調査・通信の削減、不要な情報を取得前に除く効果。

復元できる模擬アプリで、四つの仕事を行う

AppWorldの既知の開発課題を使用。要求を読ませ、ツールで実際に操作させる。

仕事	必要な工程	最後に確認すること
写真整理	一覧取得 → 日付確認 → 移動	指定どおりの配置になったか
共有・依頼	文書・連絡先・日付の確認 → 依頼	要求された処理を完了したか
誤請求への返金	請求・履歴・対象の確認 → 返金	対象の処理が正しいか
曲一覧の書出	複数の一覧取得 → 統合 → CSV作成	出力内容が要求を満たすか

モデル・要求・アプリ状態・継続予算を固定する

そろえる条件	この比較での設定
AIとツール	同じMiniMax-M3量子化モデル・同じPython／API
要求とアプリ状態	同じ要求、同じ地点まで操作済みの模擬環境
再開後の上限	16回のAI応答・12万トークン・８分のいずれかまで
API操作の上限	写真整理160回、他80回。共通の前半も数える
成功の判定	終了後に別の評価器で判定。結果をAIへ戻さない

同じ途中地点から、四条件で続きを実行する

最初から別々に走らせると、取得した情報も変わる。
そこで、４回目の応答と操作が終わった地点を共通の出発点にする。

共通の前半

1 → 2 → 3 → 4

AIの応答 → ツール実行 を４回

この時点で会話・変数・環境を保存

成功・失敗を見て地点を選ばない

- **A 全履歴＋保持** → 継続 → 評価
- **B 直近＋保持** → 継続 → 評価
- **C 全履歴＋初期化** → 継続 → 評価
- **D 直近＋初期化** → 継続 → 評価

４課題 × ４条件 × 各１回 ＝ 16実行。独立した課題数は４件。

会話の量と、変数の保持を別々に変える

	会話を全部残す 前半４回の応答＋ツール結果	会話は直近だけ 最後の１回の応答＋ツール結果
Pythonの変数を残す	A 全履歴＋保持	B 直近＋保持
Pythonを一度初期化	C 全履歴＋初期化	D 直近＋初期化

最初の要求は全条件で残す。アプリは同じ状態。

過去の取得記録は、全条件でツールから再読できる。

一つずつ変えないと、何が効いたか分からない

A と B を比較

変数を残したまま、会話だけ減らす。

保持した値があれば、履歴を減らしても継続できるか。

C と D も比較し、初期化した場合との違いを見る。

A と C を比較

同じ会話の量で、変数の保持だけ変える。

過去の会話があっても、値を残す追加価値があるか。

B と D も比較し、会話が少ない場合との違いを見る。

> 評価：予算内に仕事を終えたか ＋ 最後までに使った入力・出力と操作の量。

変数を残しても、会話短縮を補えなかった

A と B では、同じ変数を保持している。変えたのは再開時の会話の量。

課題	A 全履歴＋保持	B 直近＋保持	C 全履歴＋初期化	D 直近＋初期化
写真整理	○ **成功**	— 未達	— 未達	— 未達
共有・依頼	— 未達	— 未達	— 未達	— 未達
返金	○ **成功**	— 未達	○ **成功**	○ **成功**
曲一覧の書出	— 未達	— 未達	— 未達	— 未達
成功した課題数	**2 / 4**	**0 / 4**	**1 / 4**	**1 / 4**

> 今回の４課題・この予算では、「変数を残すから会話を減らせる」とは言えない。

入力を短くしても、総費用は増えた

A と B の比較。直近だけにすると最初の入力は短くなるが、継続全体では逆転した。

再開直後の入力

４課題の最初の１入力の合計

A 全履歴＋保持



25,528

B 直近＋保持



12,767

最初の入力は約半分

終了までの総トークン

４課題の全入力＋全出力の合計

A 全履歴＋保持



332,236

B 直近＋保持



416,022

継続全体では 25.2% 増加

単位：トークン。左右は別の尺度。実行準備の再生費用は別記。

短い履歴では、過去の情報を取り直す動きがあった

観測できたこと

前半と同じAPI要求

A 全履歴＋保持：**1回**

B 直近＋保持：**11回**

BではAPI説明・接続情報の再取得が中心。

４課題の継続部分を合計。

次に確かめる仮説

値と、値を使うための情報が結びついていない可能性

残る変数、利用できるAPI、次の作業を示すと、再取得と未達を減らせるか。

> 観測と仮説を分ける。現段階では、失敗原因も解決策の有効性も確定していない。

必要な処理を残して、何を省けるかを判断する

既存の道具・方式	研究で確かめる追加価値
履歴圧縮・選択	省いた情報の再取得まで含め、総費用を下げられるか
コード実行・状態保持・通知	既存の再利用でも残る、余分な取得・実行を減らせるか
分担・検証・停止の制御	必要な調査や確認を維持したまま、重複を避けられるか

狙う差分：不要な処理が生じる条件と、それを回避する具体的な判断規則。

現状：独自方式と改善効果は未確立。目的を見送る判断はしていない。

削減対象を一つ決め、性能維持と総費用を比べる

1

費用の発生箇所を分析

入力の再利用、再取得、呼び出し・実行の反復を分ける。

４・８応答後とも全件終了

2

省く判断を固定する

必要な処理を残せる根拠と、通常方式との差を明記する。

対象・規則は未確定

3

未調整課題で確認

品質の許容差を先に決め、制御費用込みの削減を測る。

未実施

> 新しい実験には「削減する費用・守る性能・既存の対照・採否基準」を対応させる。

費用削減の落とし穴は見えた。改善方式はこれから

ここまでの成果

同じ途中状態から比較できる基盤。
入力短縮と総費用が逆転した事例。
再取得の発生を追える実行記録。

まだ不足する証拠

性能を維持した総費用の削減。
既存方式に対する独自規則の追加価値。
未調整課題・複数担当への適用。

> 研究の主目的は、ハーネスの情報・実行制御で、性能を保ちながら総費用を減らすこと。

10月23日に、論文の主張と主要結果を絞る

10/23

主要結果・あらまし

追加比較を閉じ、既存との差を一つに絞る。

10/25

確認実験を終了

不足する対照と未調整課題の確認を優先する。

10/29–30

４ページ原稿

本文・図・引用・公式様式・著者情報を確認する。

10/31

投稿完了が目標

新規性と追加結果の成立が、現在の日程上の制約。

> 今優先すること：説明可能な一つの結果と、既存方式に対する追加価値を固める。

同じモデル・同じ環境で、構成差を比較する

計算機	Mac Studio / M3 Ultra 256 GiB RAM
モデル	MiniMax-M3 / IQ4_XS 重み約229.7 GB・版固定
推論	llama.cpp / 64K context KV q8・1スロット

環境	AppWorld 0.1.3.post1 復元可能な模擬アプリ
全体実行の上限	45万token・80生成・30分
引き継ぎ後の上限	12万token・16生成・8分
評価	終了後の別プロセス 正解をモデルへ戻さない

同じ途中状態から、全16分岐を評価する

	会話：全部	会話：直近だけ
変数を保持	● ○ ● ○ 2/4 332,236 token ／ 187 API	○ ○ ○ ○ 0/4 416,022 token ／ 196 API
変数を初期化	○ ○ ● ○ 1/4 316,491* token ／ 176 API	○ ○ ● ○ 1/4 392,787* token ／ 144 API

● 成功 ／ 淡色 未達。各 4 既知課題・1 seed。数値は後続のみ。＊：使用量不明あり（全体で 2 呼出）。表示は既知分。

> 履歴と作業状態の組合せを観測。新しい選択方式の有効性は別の検証が必要。

８応答後でも、同じ４条件・後続予算で調べる

	会話：全部	会話：直近だけ
変数を保持	● ● ● ● 1/4 324,203 token ／ 165 API	● ● ● ● 1/4 367,325* token ／ 139 API
変数を初期化	● ● ● ● 2/4 281,350 token ／ 163 API	● ● ● ● 2/4 318,722 token ／ 127 API

同じ４課題の別地点。新しい４課題ではない。後続費用のみ。＊：使用量不明あり（全体で 1 呼出）。表示は既知分。

> ２時点×16分岐でも、独立した対象は既知の４課題。一般化には別の確認が必要。

同じ４課題の全結果を、成功と費用で見る

課題	初期接続	関数説明を補足	説明＋変数保持
写真整理	成功 ／ 187,189	成功 ／ 134,387	成功 ／ 209,299
共有・依頼	成功 ／ 258,270	成功 ／ 218,993	成功 ／ 224,588
返金	成功 ／ 341,481	成功 ／ 81,974	成功 ／ 84,818
曲情報の書出	未達 ／ 177,249	未達 ／ 183,028	未達 ／ 121,330
成功数 / 全費用	**3/4 ／ 964,189**	**3/4 ／ 618,382**	**3/4 ／ 640,035**

数値＝全入力＋全出力トークン。各 1 seedの開発比較。未集計は未実行・実行中・監査待ちを含む。

> 追加の仕組みが必要かを決めるため、通常方式を十分にした状態で比べる。

P27の全比較 ／ P28の全比較 ／ P29の全比較

写真整理の残り15件は、追加操作で完了した

上限80：未達

80 API / 185,739 token



27/42

上限160：成功

97 API / 361,169 token



42/42

■ 移動の返却あり　　淡色：移動の返却なし

同じ成果物でも、完了APIの形式で合否が変わった

記録済み前半の再生	回答文あり	回答文を省略
一般指示を補う前の操作系列	未達	成功
一般指示を補った後の操作系列	未達	成功

４条件とも、完了直前のDBは同一

公開返却と保存状態のハッシュを照合。
LLM追加０、模擬API要求118。

公開仕様に沿った影響

行動依頼では回答文を省く規則がある。
評価器の不具合や独自手法の効果とはしない。